

simple_modflow

A one-page-at-a-time guide to the main workflows in the modern API.

Build, inspect, compare, calibrate, and reopen MF6 models from one consistent surface.

Build

Voronoi / DISV helpers, package builders, and a stable `SimulationBase` workflow.

Explore

Heads, budgets, package explorers, cross-sections, and surface-water summaries.

Compare

Model groups, run explorers, project catalogs, and consistent grouped plotting.

Calibrate

Reusable targets, FloPy observations, PEST workspaces, and reopenable review.

Suggested companion files

[docs/preferred_api.md](#)

[examples/mf6/notebooks/targets_api_quickstart.ipynb](#)

1. Grid & Build

Voronoi-first model setup with one durable model object at the center.

Overview & flow

The build workflow starts with geometry, not boilerplate. ``VoronoiGridPlus`` carries the spatial intelligence, while ``SimulationBase`` becomes the long-lived object that packages, plotting, and review all attach to later.

Why it matters

Treat the grid helper and the model object as the stable backbone of the whole workflow.

Suggested flow

- Use ``VoronoiGridPlus`` for polygon geometry, cached adjacency, centroid sampling, and feature-to-cell mapping.
- Attach discretization and core packages with ``DisvGrid``, ``TemporalDiscretization``, ``InitialConditions``, ``KFlow``, ``Storage``, and ``OutputControl``.
- Keep GIS inputs central with builders like ``DRNFromVector``, ``GHBFFromVector``, ``CHDFFromVector``, ``RCHFromVector``, and ``KFromVector``.
- Run once, then keep inspecting the same object through ``model.packages``, ``model.outputs``, ``model.hds``, and ``model.bud()``.

Reading notes

- ``VoronoiGridPlus`` is more than a mesh container; it is the shared geometry helper used by selection, plotting, and input builders.
- ``SimulationBase`` is the anchor object; the package-level builders mainly feed data into it rather than replacing it.

Key API surface

```
python
1  # start with a grid helper you can keep using later
2  vor = mf.VoronoiGridPlus(triangle)
3  cells = vor.get_vor_cells_as_dict(locs, loc_name_field='name')
4
5  # build one durable model object
6  model = mf.SimulationBase(
7      name='demo',
8      vor=vor,
9      nper=2,
10 )
11
12 # attach discretization and core flow properties
13 mf.DisvGrid(vor=vor, model=model, ...)
14 mf.KFlow(model=model, k=k_array, k33_vert=k33)
15 mf.Storage(model=model, sy=0.15, ss=1e-5)
16 mf.OutputControl(model=model, budget_filerecord='demo.cbb')
17
18 # run and keep using the same object
19 success, buff = model.run_simulation()
```

2. Heads & Results

Use the improved heads/results viewers for maps, tables, and sections.

Overview & flow

The results layer is meant to feel immediate: load heads once, inspect them as tables, maps, or sections, and move between spatial and tabular views without changing mental models.

Why it matters

The heads workflow should make it easy to move from raw results to a map or cross-section in one or two calls.

Suggested flow

- `model.hds`` exposes the `HeadsPlus`` helper, while `model.all_heads`` provides the normalized heads table.
- Use `model.cor(...)`` for choropleths and `model.xsect(...)`` or `plot_model_cross_section(...)`` for section views.
- Result viewers can now pivot between time selectors like `per`` and explicit `kstpkper`` pairs more consistently.
- The same model object supports both interactive review and file-backed reopened runs.

Reading notes

- The newer results surface is meant to shorten the path from file-backed heads to maps and sections.
- Because reopened runs use the same surfaces, `HeadsPlus``-style workflows carry through after calibration too.

Key API surface

```
1  # pull normalized heads from the model
2  heads = model.hds
3  table = model.all_heads.reset_index()
4
5  # map one stress period
6  fig = model.cor(per=0, layer=0, type='hds').plot()
7
8  # build a section view through selected cells
9  section = model.xsect(per=0, cells=[10, 20, 30])
10 mpl = mf.plot_model_cross_section(model, line=section.line)
11
12 # reopened runs use the same surfaces
13 head_slice = model.all_heads.query('layer == 0')
```

python

3. Targets & Observations

Define targets once, then reuse them for review, FloPy observations, and calibration.

Overview & flow

Targets are now meant to be reusable domain objects rather than a format hidden inside one notebook or one calibration script. Define them once, then feed them everywhere else.

Why it matters

The same target definition should support comparison, plotting, FloPy observation files, and PEST.

Suggested flow

- `HeadTargets` supports GIS points, cell dictionaries, `Series`, record tables, and `from_cells(...)` / `from_records(...)` helpers.
- `LakeStageTargets` supports lake-id mappings, `Series`, records, and plain lists with `times=...`.
- Attach them to the model with `model.targets.heads = ...` and `model.targets.lake_stage = ...`.
- Generate MF6 observation objects directly with `to_flopy_obs(...)` and `attach_flopy_obs(...)`.

Reading notes

- The plain constructors now accept lightweight dict and series inputs, while the classmethods remain useful for more structured cases.
- The same target object can stay attached to the model and still generate lower-level FloPy observation packages when needed.

Key API surface

```
1  # attach reusable head targets
2  model.targets.heads = mf.HeadTargets(
3      locations={'OBS_01': 101},
4      values=series,
5      time_column='per',
6  )
7
8  # lake stages can come from simple sequences too
9  model.targets.lake_stage = mf.LakeStageTargets(
10     locations={'lake_main': 0},
11     values=[766.1, 766.3],
12     times=[0, 1],
13 )
14
15 obs_pkg = model.targets.heads.attach_flopy_obs(
16     filename='head_targets.obs',
17     csv_name='head_targets.csv',
18 )
19
20 compare = model.targets.heads.compare()
21 stats = model.targets.heads.stats()
```

python

4. Budgets & Package Results

Use the improved budget wrappers and package explorers for tabular and map views.

Overview & flow

Budget inspection should be as direct as head inspection. The newer budget surfaces wrap both listing-style summaries and package-level results so you can stay in one API while moving from totals to cells.

Why it matters

Use the top-level budget helper for general accounting, then drop into package explorers for cell-by-cell detail.

Suggested flow

- `model.bud()` returns the improved `'Budget'` wrapper, while `model.bud('rch')` or similar scopes the view to one package.
- Package explorers now expose `get()`, `map()`, `plot_timeseries()`, and `plot_budget()` more consistently across LAK, SFR, and other namespaces.
- Budget viewers and result explorers now share more predictable period selectors and normalized result tables.
- Use the package namespaces when you want connection-aware or term-aware output rather than one flat budget table.

Reading notes

- The modern budget workflow tries to make totals, tables, and package detail feel connected instead of like separate tools.
- When you need richer package semantics, the package explorers are usually the better surface than the raw listing budget.

Key API surface

```
1  # work from the top-level budget helper
2  gwfbudget = model.bud()
3  rchbudget = model.bud('rch')
4
5  # listing-style summaries stay easy to reach
6  summary = gwfbudget.budget()
7  plot = rchbudget.plot_budget_obs(obs_dict)
8
9  # package explorers go deeper
10 lak_q = model.packages.lak.results.q.get(per=0)
11 model.packages.sfr.results.q.plot_budget()
12 model.packages.lak.budget.gwf.summary()
```

python

5. Surface Water

Lakes, streams, and combined exchange now have clearer result and validation surfaces.

Overview & flow

Surface-water diagnosis is no longer split across a handful of isolated helpers. Lakes, streams, and combined exchange can now be inspected through a more consistent package/results surface, with validation close by.

Why it matters

The surface-water API should make it easy to move from one package to a combined system view without changing conventions.

Suggested flow

- LAK and SFR package explorers now expose normalized result tables and budget-term helper namespaces.
- Combined `surface\_water` exchange maps use a shared sign convention and shared scaling so lakes and streams can be read together.
- `model.validate\_surface\_water()` and related validators are meant to catch configuration issues before they become interpretation problems.
- Grouped surface-water subplots carry the same ideas into scenario comparison.

Reading notes

- The package-level helper namespaces matter because lake-stage, stream exchange, and routing terms do not all live at the same physical level.
- The combined `surface\_water` view is often the fastest way to spot where the system is gaining or losing water overall.

Key API surface

```
1  # inspect lake and stream results directly
2  stage = model.packages.lak.results.stage.get()
3  model.packages.sfr.results.q.map(per=0)
4
5  # use the combined exchange view when needed
6  model.packages.surface_water.results.q.map(per=0)
7
8  # validate configuration before deeper review
9  report = model.validate_surface_water()
10
11 # package budget helpers stay available too
12 model.packages.sfr.budget.flow_ja_face.get(per=0)
```

python

6. Model Groups

Compare multiple runs through one lazy, aligned API.

Overview & flow

Scenario comparison should not require custom merge logic every time. `ModelGroup` is there to align models, preserve a reference run, and expose grouped heads, grouped packages, and grouped surface-water plots from one surface.

Why it matters

Use groups when comparison is the primary task, not as an afterthought after single-model analysis.

Suggested flow

- `ModelGroup` accepts model objects or run/workspace paths and can lazily open file-backed runs for comparison.
- Use `group.hds.get()` and `group.hds.compare()` for aligned head tables and reference-based differences.
- Grouped package accessors now support side-by-side maps and plots for inputs and results.
- Use `shared_grid=True` when models share the same discretization so the grid view can be reused efficiently.

Reading notes

- The grouped API is most useful when you want one reference model and one or more alternatives to stay aligned automatically.
- Grouped surfaces are meant to reuse the same ideas as the single-model surfaces, not invent a parallel dialect.

Key API surface

```
python
1 # build a comparison group
2 group = mf.ModelGroup(models, reference='base', shared_grid=True)
3
4 # compare heads against the reference model
5 head_diff = group.hds.compare(per=0)
6
7 # build grouped package maps
8 group.packages.surface_water.results.q.subplot_map(per=0)
9 group.packages.lak.results.stage.plot_timeseries('lake_main')
10
11 # summarize the participating runs
12 group.summary()
```

7. Project & Run Management

The same codebase also supports archive-style and project-managed workflows.

Overview & flow

Not every useful model starts inside one notebook. Some are long-lived archives, some are organized around related scenarios, and some need richer artifact lineage. The project layer is there for that broader lifecycle.

Why it matters

Use the lightweight exploration path for existing runs, and the catalog path when you want managed scenario workflows.

Suggested flow

- Use ``ProjectCatalog``, ``RunSpec``, and artifact helpers when you want managed related runs and reproducible scenario lineages.
- Use ``load_mf6_run(...)`` and ``RunExplorer`` when you want to lazily inspect existing archives without rebuilding models.
- The long-term aim is a smooth path from model build to calibration to results review without leaving the same conceptual API.
- Notebook and script examples in ``examples/mf6/`` are the fastest way to see those patterns together.

Reading notes

- Use the lazy loader when you want to inspect existing run folders without rebuilding anything first.
- Use the project/catalog path when scenario lineage and artifacts matter more than ad hoc experimentation.

Key API surface

```
1  # lazy-open an existing MF6 archive
2  loaded = mf.load_mf6_run(workspace)
3  explorer = mf.RunExplorer.from_directory(root)
4
5  # organize related runs when lineage matters
6  catalog = mf.ProjectCatalog(root=project_root)
7  records = explorer.records
8
9  # reopen a completed calibration run
10 run = mf.open_pest_run(pest_workspace)
```

python

8. Plotting & Review

Targets, calibration plots, and model-bound review surfaces stay close together.

Overview & flow

Plotting is easier to discover when it lives next to the thing being plotted. The newer model-bound target surfaces are meant to shorten the path from a target definition to maps, cross-plots, and time-series review.

Why it matters

Use `model.targets...` for the common review calls, and let the lower-level plotting tools stay secondary.

Suggested flow

- Use `model.targets.heads.plot.locations()` for target maps and `plot.obs_vs_sim()` / `plot.timeseries(...)` for review.
- The same targets can generate `CalibrationPlot` views without re-ingesting spreadsheets or shapefiles.
- This target layer is intentionally independent of PEST, even though the calibration workflow also consumes it.
- Lake-stage targets and head targets can both stay attached to the model while still generating FloPy observation packages.

Reading notes

- The long-term role of `CalibrationPlot` is to consume the target system rather than act as a separate ingestion layer.
- If a review workflow still needs custom joins or one-off plotting glue, that is usually an API smell worth fixing.

Key API surface

```
python
1  # compare target heads against one model
2  compare = model.targets.heads.compare()
3  stats = model.targets.heads.stats()
4
5  # stay close to the target object while plotting
6  model.targets.heads.plot.locations()
7  model.targets.heads.plot.obs_vs_sim()
8
9  # transient review stays one call away
10 model.targets.heads.plot.timeseries('OBS_01')
11
12 plot = mf.CalibrationPlot.from_targets(model.targets.heads)
13 mf.CalibrationPlot.from_compare(compare)
```

9. Calibration / PEST

The reusable pyEMU layer is designed to sit on top of the general model workflow.

Overview & flow

Calibration should feel like an extension of the model workflow, not a separate universe. Parameter specs and observation specs should read like structured inputs, while saved metadata should make the resulting runs easy to reopen later.

Why it matters

Keep target definitions general and let the PEST layer adapt them instead of redefining them.

Suggested flow

- Create a `PestProject(model=..., workspace=...)` and add parameter specs such as `KPilotPointParameter` and `DrainConductanceParameter`.
- Use `HeadTargetObservationSpec(targets=model.targets.heads.targets)` so the calibration layer consumes the shared target object.
- Built workspaces save normalized metadata and target snapshots so completed runs can be reopened automatically later.
- Result review is meant to flow back into the normal model/review API rather than staying trapped in the PEST workspace.

Reading notes

- The calibration layer adapts model-side targets rather than redefining them, which keeps the mental model cleaner.
- Saved run metadata is what later allows completed PEST workspaces to reopen smoothly through the review API.

Key API surface

```
python
1 # build a reusable calibration workspace
2 pest = mf.PestProject(
3     model=model,
4     name='calibration_run',
5     workspace=model.workspace / 'pest',
6     start_datetime='2024-01-01',
7 )
8
9 pest.add_parameter(mf.KPilotPointParameter(...))
10 pest.add_observation(
11     mf.HeadTargetObservationSpec(
12         targets=model.targets.heads.targets,
13     )
14 )
15
16 pst = pest.build_pst('run.pst')
17 pst.control_data.noptmax = 3
```

10. Reopen & Reuse

Completed calibration workspaces should come back as normal reviewable model objects.

Overview & flow

A calibrated run is most useful when it comes back as a normal model object. The post-run review path should feel like continuing the same analysis, not starting over from a folder full of artifacts.

Why it matters

Reopened calibrated models should be first-class `SimulationBase`-style objects, not just piles of files on disk.

Suggested flow

- Open a completed workspace with `mf.open_pest_run(...)` and load the baseline model, calibrated model, and saved targets.
- `load_calibrated_model()` reapplies final parameters and materializes fresh MF6 outputs before comparison.
- Use `review()`, `compare_head_targets()`, `plot_k()`, `plot_k_ratio()`, and `plot_well_timeseries(...)` for post-run diagnosis.
- This same reopen surface is what lets the broader project and notebook workflows stay smooth after long runs complete.

Reading notes

- The reopened calibrated model is meant to behave like a normal `SimulationBase`-style object, not just a file bundle.
- The cleanest workflow is the one that keeps the same concepts alive from first build to final review.

Key API surface

```
1  # reopen a completed calibration workspace
2  run = mf.open_pest_run(workspace)
3
4  # load baseline and calibrated model states
5  baseline = run.load_baseline_model()
6  calibrated = run.load_calibrated_model()
7  targets = run.load_head_targets()
8
9  # collect common review products
10 review = run.review()
11 run.plot_k_ratio()
12 run.compare_head_target_stats()
```

python